

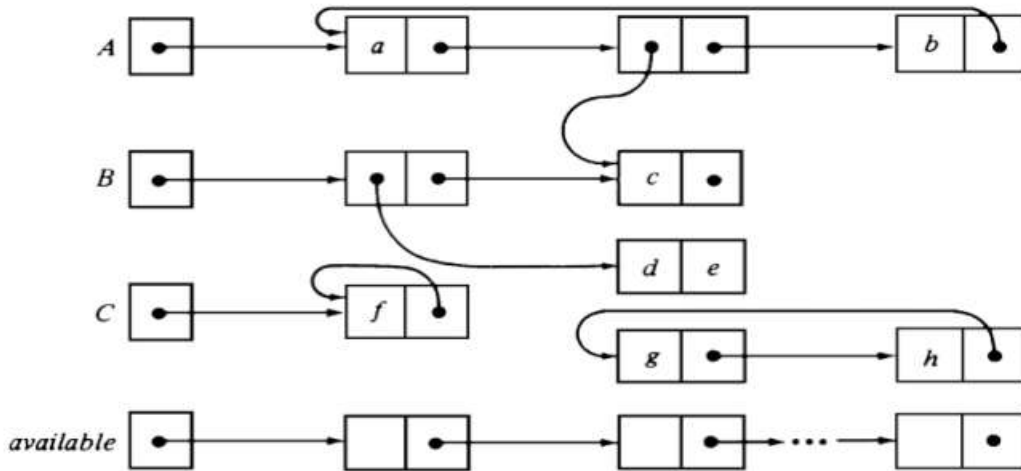
Unit: II
Lecture: 6
Garbage Collection Algorithms for equal sized blocks

For equal sized blocks, let's consider the example of programming language LISP.

In LISP, memory space is divided into cells, which are essentially records consisting of two fields: each field can hold either an atom (object of elementary type e.g. an integer) or a pointer to a cell.

Atoms and pointers are the same size, so all cells require same number of bytes.

Empty cells are kept on an available space list. An atom is mostly pointed by one more pointers. This forms a network of cells like shown below:



In figure A,B,C are variables and a,b,c,d,e,f,g,h are atoms. One atom can be pointed by more than one pointer like a,c,f; each cell is reachable by one of the variable.

But g, h are unusual as they are not reachable from any of the variable ABC but do have a pointer pointing towards them. So, they are not in the available space list.

Ever Changing network of cell:

The pointer may get changes from time to time. When variables need more cell, it takes one from the available space list and the pointer changes.

Also, when the cell gets pointed by no one then it is added to the available space list and again pointers get changed. Sometimes, the values of the variable change so they need to point to some other atom. In that case also the pointer changes and network of cells keep changing itself.

Need of Garbage Collection:

Due to ever changing nature of cell network when cell are detached and therefore are no longer needed by the program, it would be nice if they found their way back to available space list. This way they could be reused by the program on a later stage. If such cells are not reclaimed they will remain unused and sooner or later the program shall run out of empty cells.

→ a time consuming garbage collection must be performed.

Importance of garbage collection:

- Minimizes the time spent reclaiming memory.
- Minimizes the amount of wasted memory at all times.
- Minimizes the amount of memory necessary to perform a collection.
- Minimizes the time and resources necessary for the program to access the memory during normal execution (including maximizing CPU cache hit rate and minimizing OS page faults).
- Minimizing the pause-time experienced by an application during a collection.

Garbage Collection algorithm for equal sized blocks:

Any garbage collection algorithm must perform 2 basic operations:

- It should be able to detect all the unreachable objects and
- secondly, it must reclaim the heap space used by the garbage objects and make the space available again to the program.

There are four variants of cells which are:

- PP → both cells contains the pointer.
- PA → left field is a pointer and the right field an atom.
- AF → left field is an atom and the right field is a pointer.
- AA → both cells are atoms.

An additional field called mark, indicate whether the cell has been found accessible.

- Setting mark as true means the cell is accessible from the source.
- Setting mark as false means the cell is not accessible from the source.

To mark the cell accessible from the source:

- Firstly unmark all the cells or set mark as false.
- Then perform depth first search starting from the source marking all cells visited i.e. setting mark as true.
- Finally add the leftover unmarked cells to the available space list.

The algorithm uses two procedures:

- COLLECT: to unmark all cells
- DFS: to perform dept first search

ALGORITHM marking_accessible_cells

CALL COLLECT

CALL DFS

PROCEDURE COLLECT

Initialize i=1

While (i < memorysize)

Mark = false

PROCEDURE DFS

IF mark = false THEN

Mark= true

IF (pattern=PP) or (Pattern=PA) THEN

IF left != Nil THEN

DFS(left)

IF (pattern=PP) or (Pattern=AP) THEN

IF right != Nil THEN

DFS(right)